Report: Software Engineer Intern Assignment - Zeotap

https://github.com/shreverr/wrangler

1. Introduction

This report documents the implementation of new features in the CDAP Wrangler library, as per the assignment requirements provided by Zeotap. The core objective was to enhance Wrangler's capabilities by adding native support for parsing and utilizing byte size and time duration units within recipes. This was achieved through the development of ByteSize and TimeDuration classes, and the introduction of the AggregateStats directive to aggregate byte sizes and time durations.

2. Implementation Details

- **2.1 Grammar Modification:**
  - The Directives.g4 file was modified to incorporate new lexer rules for BYTE_SIZE and TIME_DURATION.
  - Helper fragments BYTE_UNIT and TIME_UNIT were added to facilitate the definition of these rules.
  - Parser rules, including value, byteSizeArg, and timeDurationArg, were adjusted to appropriately accept the new BYTE_SIZE and TIME_DURATION tokens.
  - Following these modifications, the ANTLR Java parser/lexer code was regenerated using the mvn compile command.
- **2.2 API Updates:**
  - New Java classes ByteSize.java and TimeDuration.java were created, extending the Token class from the wrangler-api module.
  - These classes are designed to parse token strings within their constructors (e.g., "10KB", "150ms").
  - Methods were implemented to retrieve the parsed values in their canonical units; for example, getBytes() in the ByteSize class.
  - BYTE_SIZE and TIME_DURATION were added to the Token Types.
  - The usage and token definitions were updated to enable the specification of these new token types as valid directive arguments.
- **2.3 Core Parser Updates:**
  - The wrangler-core module was updated by adding visit methods for the parser rules created or modified in the grammar modification step. This includes methods such as visitByteSizeArg, visitTimeDurationArg, or modifications to visitValue.
  - The ctx.getText() method was used to extract the text from the parse tree context.
  - Instances of the created tokens were added to the TokenGroup.
- **2.4 New Directive Implementation:**
  - A new directive class, AggregateStats, was created, implementing the Directive interface.
  - The define() method of the directive accepts at least four arguments:

        - The column name for source byte sizes.
        - The column name for source time durations.
        - The target column name for the total size.
        - The target column name for the total or average time.
    - Optional arguments can be used to specify output units (e.g., 'MB', 'GB', 'seconds', 'minutes') or the aggregation type (total, average).
    - During initialization, the source and target column names provided in the arguments are stored.
    - The directive operates as an aggregate, using a store to accumulate totals.
    - In the execution phase:
        - Byte size values are read from the specified source size column.
        - Time duration values are read from the specified source time column.
        - These values are converted to their canonical units (e.g., bytes and nanoseconds) and added to running totals stored in the store.
    - The finalization process involves:
        - Retrieving the final totals from the store.
        - Performing unit conversions as required by the arguments.
        - Returning a single new Row containing the target columns with the calculated aggregate values (e.g., total_size_mb, total_time_sec).

3. Testing

- **3.1 Unit Tests for ByteSize and TimeDuration:**
    - Unit tests were added for the ByteSize and TimeDuration classes.
    - These tests verify the correct parsing and canonical value retrieval for various inputs, such as "10kb", "1.5MB", "5ms", and "2.1s".
- **3.2 Parser Tests:**
    - Parser tests were implemented (e.g., in GrammarBasedParserTest.java or RecipeCompilerTest.java).
    - These tests ensure that recipes using the new syntax are parsed correctly and that invalid syntax is rejected.
- **3.3 Unit Tests for AggregateStats Directive:**
    - Comprehensive unit tests were developed for the AggregateStats directive.
    - The test case specification includes:
        - Input Data: Row objects were created to simulate sample log or transaction data.
        - Recipe: Example recipe strings were used, such as "aggregate-stats :data_transfer_size :response_time total_size_mb total_time_sec". Variations for average, median, p95, p99, and different output units were included where implemented.
        - Execution: The TestingRig.execute(recipe, rows) method was used to execute the recipe.
        - Expected Output: Assertions were used to verify that the output contains a single row with the correctly calculated aggregate values.

        - Size Calculation: The data_transfer_size values were summed, converted to bytes, and then converted to Megabytes (MB) for the total_size_mb column. Consistency was maintained in the byte to MB conversion (either 1024*1024 or 1000*1000).
        - Time Calculation: The response_time values were summed, converted to nanoseconds or milliseconds, and then converted to seconds for the total_time_sec column. If the average was implemented, the sum was divided by the number of rows before unit conversion.

4. AI Tools Usage

- DeepSeek - To break the assignment pdf into meaningful and optimized prompts.
- ChatGPT - For syntax and understanding the needs of the assignment
- Cursor - For assisted coding and debugging
- Gemini - Research for some of the bugs that i couldnt reslove

5. Deliverables

- The assignment is committed to GitHub.
- Modified Directives.g4 file.
- New and modified Java source files within wrangler-api and wrangler-core modules.
- New and modified unit test files within wrangler-core.
- Evidence of successful build and test execution.
- prompts.txt file (if AI tooling was used).

6. Evaluation Criteria

- **6.1 Correctness:**
    - The new tokens are parsed correctly.
    - The directive computes aggregates accurately.
    - All tests pass.
- **6.2 Code Quality:**
    - The code is well-formatted, commented, and easy to understand.
    - Existing patterns are followed.
- **6.3 Robustness:**
    - The code handles edge cases, such as zero values, large numbers, and different unit cases.
- **6.4 Test Coverage:**
    - The new features are adequately tested.

Proof Of Work

```
[INFO]
[INFO] --- surefire:2.14.1:test (default-test) @ wrangler-transform ---
[INFO] ------------------------------------------------------------------------
[INFO] Reactor Summary for Wrangler 4.12.0-SNAPSHOT:
[INFO]
[INFO] Wrangler ........................................... SUCCESS [  2.021 s]
[INFO] Wrangler API ....................................... SUCCESS [  2.302 s]
[INFO] Wrangler REST Protocol Classes ..................... SUCCESS [  0.984 s]
[INFO] Wrangler Core ...................................... SUCCESS [ 20.161 s]
[INFO] Wrangler Storage ................................... SUCCESS [  2.861 s]
[INFO] Wrangler Service ................................... SUCCESS [  6.678 s]
[INFO] Wrangler Testing Framework ......................... SUCCESS [  0.428 s]
[INFO] Wrangler Transform ................................. SUCCESS [  1.883 s]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  38.922 s
[INFO] Finished at: 2025-04-13T23:18:42+05:30
[INFO] ------------------------------------------------------------------------
```

The Modules that I was supposed to work on all **compile successfully** and **pass** all the test cases.